



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ICS2123 – MODELOS ESTOCÁSTICOS
PROFESORES NICOLÁS VARGAS Y MAXIMILIANO GONZÁLEZ
AYUDANTE JEFE FRANCISCO TAMARÍN
PRIMER SEMESTRE 2025

Tarea 4: *Sistemas de Espera y Simulación*

Integrantes:

- ◇ Denisse Alejandra Rojas Burgos, Sección 4
- ◇ Franco Nicolai Corvalán Mansourova, Sección 4

Instrucciones

- ◇ Esta tarea debe ser realizada **individualmente** o **en parejas** (pueden ser de diferentes secciones dentro de las que son coordinadas). Deben indicar en la primera plana del PDF los nombres de los/as integrantes, junto con las secciones respectivas de c/u.
- ◇ Tienen plazo hasta el **miércoles 25 de junio a las 23:59 hrs** para entregar sus soluciones.
- ◇ Deben entregar un archivo reporte autocontenido, **en formato PDF**. En el archivo PDF deben entregar sus respuestas de los 3 problemas. Los códigos implementados deben adjuntarse como un archivo **.ZIP aparte** en el buzón. En caso de no cumplir con esto, se aplicará un descuento de **5 décimas** sobre la nota final de la tarea.
- ◇ En el PDF deberá estar el desarrollo, resumen de resultados y correspondiente análisis de **todas las preguntas**. Se recomienda fuertemente utilizar $\text{\LaTeX}$. Sin embargo, estas también pueden ser escritas en algún otro editor de texto o a mano y escaneadas, pero **debe estar ordenado y legible**, de lo contrario, se descontarán **5 décimas** a la nota final de la tarea. Cualquier código o referencia que utilicen de internet debe ser debidamente citado insertando el *link* asociado.
- ◇ Si realizan la tarea completa en $\text{\LaTeX}$, incluyendo todas las fórmulas y ecuaciones, tendrán **3 décimas extras**.
- ◇ La tarea tiene un total de 60 puntos. La nota final será calculada como la suma de las tres preguntas mediante la fórmula

$$Nota = \frac{P_1 + P_2 + P_3}{60} \cdot 6 + 1 + Bonus - Descuentos$$

Donde P_i corresponde al puntaje de la pregunta i . Se aproxima a dos decimales.

- ◇ Las preguntas se harán a través del foro de discusión asociado a la Tarea 4 en Canvas, por lo que les pedimos que **NO** manden dudas por mail sobre la tarea. De esta manera, todos tienen acceso a las respuestas.
- ◇ Seremos estrictos en que **no se aceptarán envíos por mail**. Evite problemas de internet, del sistema Canvas, de la calidad de la foto o cualquier otro que pudiese ocurrir al entregar sobre la hora. El/La alumno/a que envíe su tarea después de la hora límite, tendrá **nota 1.0**.

Problema 2

Ítem a

Las variables de estado identificadas corresponden a:

- ◇ T : tiempo actual de simulación, debe ser $\leq T_{limite}$
- ◇ N_{cola} : número de pedidos en cola.
- ◇ $status_1$: 1 si el centro 1 está ocupado, 0 si está libre.
- ◇ $status_2$: 1 si el centro 2 está ocupado, 0 si está libre.
- ◇ $N_{clientes_1}$: numero de pedidos procesados por centro 1.
- ◇ $N_{clientes_2}$: numero de pedidos procesados por centro 2.
- ◇ $T_{llegada}$: tiempo hasta la próxima llegada (distribución D).
- ◇ T_{salida_1} : tiempo de los próximos términos de atención (distribuciones H1).
- ◇ T_{salida_2} : tiempo de los próximos términos de atención (distribuciones H2).
- ◇ T_{limite} : tiempo máximo de simulación
- ◇ P_{max} : máximo de pedidos totales
- ◇ $ocupacion_1$ y $ocupacion_2$: acumuladores de tiempo ocupado

Por otro lado, los eventos que pueden ocurrir durante la operación del sistema son los siguientes:

- ◇ Salida de productos desde el centro 1.
- ◇ Salida de productos desde el centro 2.
- ◇ Ingreso de productos al centro 1.
- ◇ Ingreso de productos al centro 2.

Ítem b

El diagrama de flujo construido corresponde al siguiente:

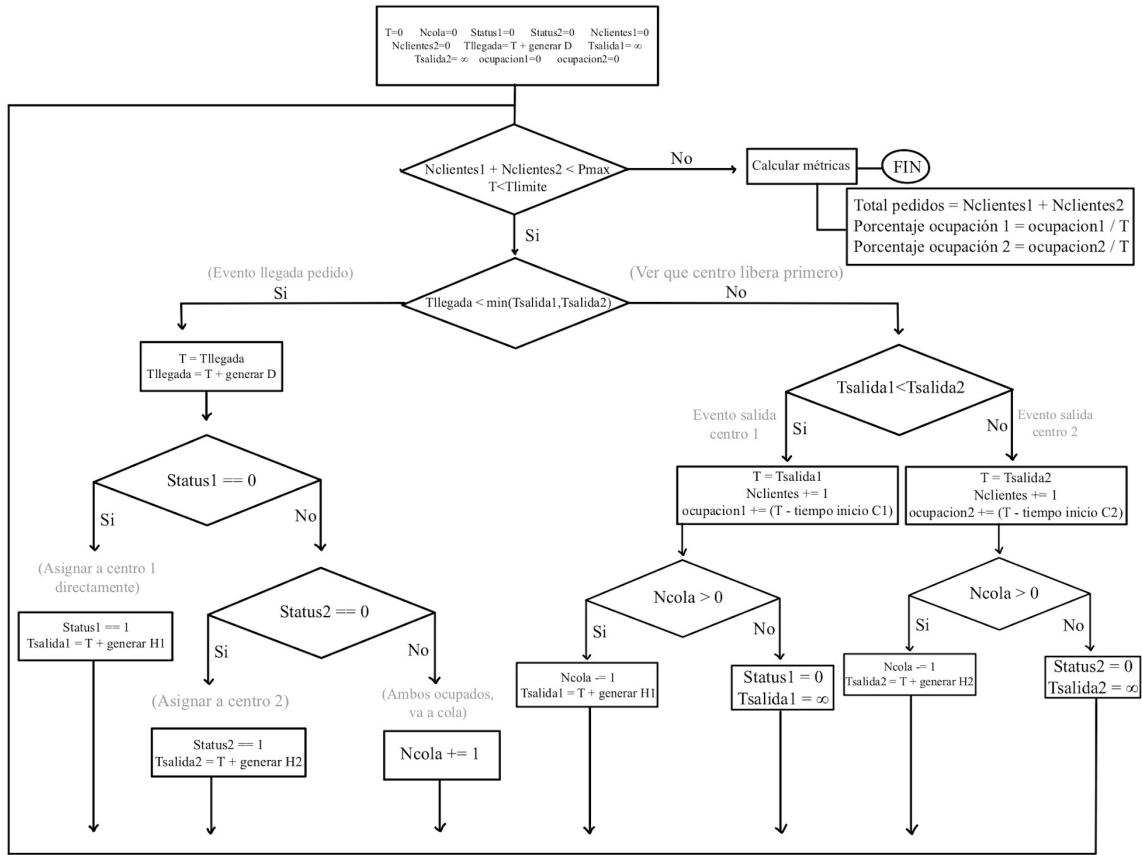


Figura 1: Diagrama de flujo

Problema 3

Ítem a

Considerando el método de congruencia lineal (LCG) para la generación de números aleatorios con distribución uniforme $U(0, 1)$, y utilizando los distintos conjuntos de parámetros entregados en el Cuadro 1, se implementó el siguiente algoritmo:

- ◊ Se elige un valor inicial X_0 (semilla).
- ◊ Se genera la secuencia mediante la recurrencia:

$$X_i = (aX_{i-1} + c) \text{ mód } m$$

- ◊ Los números aleatorios uniformes se obtienen como:

$$U_i = \frac{X_i}{m}$$

El código utilizado fue el siguiente:

```
def metodo_congruencia(m, a, c):
    x = 123
    lista_valores_u = []
    lista_valores_x = [x]

    while True:
        x = (a * lista_valores_x[-1] + c) % m

        if x not in lista_valores_x:
            lista_valores_x.append(x)
```

```

        u = x / m
        lista_valores_u.append(u)
    else:
        break

    return len(lista_valores_u)

sets_de_parametros = {
    "Set 1": {"m": 10007, "a": 23, "c": 17},
    "Set 2": {"m": 10503, "a": 89, "c": 1},
    "Set 3": {"m": 10021, "a": 114, "c": 37}
}

print("--- Generando secuencias de nmeros pseudoaleatorios ---")
print(f"\nSet 1: {metodo_congruencia(**sets_de_parametros['Set 1'])}")
print(f"Set 2: {metodo_congruencia(**sets_de_parametros['Set 2'])}")
print(f"Set 3: {metodo_congruencia(**sets_de_parametros['Set 3'])}\n")

```

Los resultados obtenidos fueron:

- ◊ Set 1: 10005
- ◊ Set 2: 1163
- ◊ Set 3: 454

A partir de los resultados obtenidos, podemos evaluar el desempeño de cada conjunto de parámetros en función de la cantidad de números pseudoaleatorios distintos generados antes de repetir el ciclo (longitud del período), concluyendo que en particular el set 1 presenta el mejor desempeño, logrando generar 10005 números antes de repetir la secuencia. Esto es debido a que un buen generador congruencial debe presentar períodos lo más largos posibles, para garantizar una buena cobertura del intervalo $U(0,1)$ y una mayor independencia aparente entre los valores generados.

Ítem b

Considere una variable aleatoria continua cuya función de densidad de probabilidad $f(x)$ está definida por tramos como:

$$f(x) = \frac{1}{Z} \begin{cases} \lambda_1 e^{-\lambda_1 x}, & 0 \leq x < 1, \\ \lambda_2 e^{-\lambda_2(x-1)}, & 1 \leq x < 2, \\ \vdots \\ \lambda_n e^{-\lambda_n(x-[n-1])}, & n-1 \leq x < n, \end{cases}$$

donde $Z > 0$ es una constante de normalización.

Para generar muestras de esta variable aleatoria utilizando el método de la transformada inversa, debemos calcular la inversa de su función de distribución acumulada y evaluarla en una variable uniforme. Es decir, considerando el tramo general:

$$f(X) = \frac{1}{Z} \lambda_n e^{-\lambda_n(x-[n-1])}.$$

Calculamos su inversa para obtener:

$$F^{-1}(U).$$

Como resultado, obtenemos:

$$X = -\frac{1}{\lambda_n} \ln \left(\frac{Z \cdot U}{\lambda_n} \right) + n + 1,$$

donde Z es la constante de normalización y $U \sim \text{Uniforme}(0,1)$ o sea, U es una transformación de la uniforme para abarcar el dominio de $f(x)$.

Es importante mencionar que esta es una función definida por tramos, por lo que sus componentes dependerán del dominio.

Luego, consideraremos el caso particular en que $n = 5$ y $\lambda_i = i$ para todo $i \in \{1, 2, 3, 4, 5\}$, generaremos la mayor cantidad posible de instancias de esta variable aleatoria.

Con lo anterior, se desarrollará el código para validar el método propuesto.

```
import numpy as np
import math
import matplotlib.pyplot as plt

# --- Parametros ---
n = 5
lambdas = np.array([1,2,3,4,5])
cantidad_muestras = 10000
seteo_de_semilla = 2123

# --- Semilla ---
np.random.seed(seteo_de_semilla)

# --- Funcion_acumulada ---
def f():
    suma_integrales_sin_C = 0
    for i in range(0, n):
        lambda_i = lambdas[i]
        suma_integrales_sin_C += (1 - np.exp(-lambda_i))

    return suma_integrales_sin_C

# --- Obtenemos constante de normalizacion ---
f_acumulada = f()
Z = 1/f_acumulada
print(f"Valor de contante de normalizacion: {Z}")

# --- Generamos muestras mtodo funcin inversa ---
def generar_muestras():
    muestras = []
    for _ in range(cantidad_muestras):
        # Elegimos primero el tramo aleatoriamente
        tramo = np.random.choice(n)
        lambda_i = lambdas[tramo]

        # Aplicamos la inversa de la exponencial en ese tramo
        u = np.random.uniform(0, 1)
        x = tramo - np.log(1 - u) / lambda_i

        muestras.append(x)
    return np.array(muestras)

# --- Funcion ---
def f(x):
    piso = int(np.floor(x))
    if piso >= n:
        return 0
    lambda_i = lambdas[piso]
    return Z * (lambda_i * np.exp(-lambda_i * (x - piso)))

# --- Densidad teorica ---
def densidad_teorica():
    x_vals = np.linspace(0, n, 1000)
    f_vals = [f(x) for x in x_vals]
    return x_vals, f_vals

# --- Histograma ---
```

```
def histograma(resultados, x_vals, f_vals):
    plt.figure(figsize=(20,12))
    plt.hist(resultados, bins=50, density=True, alpha=0.6, color='skyblue', label='Histograma de
        muestras')
    plt.plot(x_vals, f_vals, 'r', lw=2, label='Densidad terica')
    plt.title('Ajuste emprico vs. densidad terica')
    plt.xlabel('x')
    plt.ylabel('Densidad')
    plt.legend()
    plt.show()

if __name__ == "__main__":
    resultados = generar_muestras()
    x_vals, f_vals = densidad_teorica()
    histograma(resultados, x_vals, f_vals)
```

Al ejecutar el código anterior, obtenemos el siguiente histograma que muestra los resultados obtenidos:

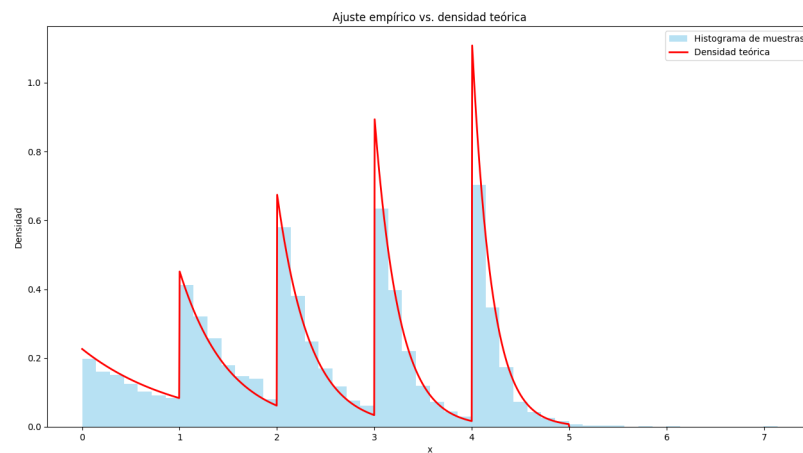


Figura 2: Histograma de los valores generados mediante el método de transformada inversa

Analice detalladamente los resultados obtenidos: comente el ajuste entre la distribución empírica y la teórica, identifique posibles sesgos, y determine en qué intervalos se concentra la mayor densidad de probabilidad. Finalmente, discuta la eficiencia del método de generación implementado.

Ítem c

Considere una variable aleatoria continua cuya función de densidad de probabilidad $f(x)$ está definida por tramos como:

$$f(x) = \begin{cases} cxe^{-x}, & \text{si } 0 \leq x \leq 2, \\ c(4-x)e^{-x}, & \text{si } 2 < x \leq 4, \\ 0, & \text{en otro caso,} \end{cases}$$

donde $c > 0$ es una constante de normalización.

Para obtener la constante de normalización, consideramos que como $f(x)$ es una función de densidad de probabilidad, su integral en todo el dominio debe ser igual a 1. Es decir:

$$\int_0^4 f(x) dx = 1.$$

Esto se puede escribir como:

$$\int_0^2 cxe^{-x}dx + \int_2^4 c(4-x)e^{-x}dx = 1.$$

Calculando las integrales:

$$\int_0^2 cxe^{-x}dx = c[-(x+1)e^{-x}]_0^2,$$

y

$$\int_2^4 c(4-x)e^{-x}dx = c[(x-3)e^{-x}]_2^4.$$

Por lo tanto:

$$1 = c(-(2+1)e^{-2} + (0+1)e^0 + (4-3)e^{-4} - (2-3)e^{-2}).$$

Simplificando y calculando numéricamente, se obtiene:

$$c == 0.33438326449781075$$

Luego, generaremos las condiciones para aplicar el algoritmo de Aceptación-Rechazo.

Primero, calculamos los máximos valores de $f(x)$ en cada tramo, derivando cada parte de la función por separado:

◇ Para el primer tramo $0 \leq x \leq 2$:

$$\frac{d}{dx}(cxe^{-x}) = c(e^{-x} - xe^{-x}) = ce^{-x}(1-x).$$

Igualando a cero:

$$1-x=0 \Rightarrow x_{\text{máx}}=1.$$

Evaluando en este punto:

$$f(1) = c \cdot 1 \cdot e^{-1} = \frac{c}{e}.$$

◇ Para el segundo tramo $2 < x \leq 4$:

$$\frac{d}{dx}(c(4-x)e^{-x}) = c(-e^{-x}(1+(4-x))) = ce^{-x}(3-x).$$

Igualando a cero:

$$3-x=0 \Rightarrow x_{\text{máx}}=3.$$

Evaluando en este punto:

$$f(3) = c(4-3)e^{-3} = \frac{c}{e^3}.$$

Por lo tanto, el máximo global de $f(x)$ en su dominio ocurre en $x=1$, con valor:

$$f(1) = \frac{c}{e}.$$

Si reemplazamos el valor numérico de $c \approx 1.337$, obtenemos:

$$f(1) \approx \frac{1.337}{e} \approx 0.491.$$

Finalmente, definimos la función propuesta $g(x)$ para el método de Aceptación-Rechazo. Como el dominio de $f(x)$ es el intervalo $[0, 4]$, la función uniforme asociada es:

$$g(x) = \frac{1}{4-0} = \frac{1}{4}.$$

Luego, planteamos la condición de aceptación-rechazo, que requiere que:

$$\mu \cdot g(x) \geq f(x).$$

Dado que $g(x)$ es uniforme en $[0, 4]$, tenemos:

$$g(x) = \frac{1}{4}.$$

Adicionalmente, como $f(x)$ está definido por tramos, evaluamos la constante μ en ambos tramos.

Tramo 1: $0 \leq x \leq 2$

$$\mu \cdot \left(\frac{1}{8}\right) \geq \frac{c}{e} \quad \Rightarrow \quad \mu \geq \frac{8c}{e}.$$

Tramo 2: $2 < x \leq 4$

$$\mu \cdot \left(\frac{1}{8}\right) \geq \frac{c}{e^3} \quad \Rightarrow \quad \mu \geq \frac{8c}{e^3}.$$

Como buscamos la cota más exigente (la mayor), finalmente:

$$\mu \geq \frac{4c}{e}.$$

Luego, para poder utilizar el algoritmo, transformamos la variable uniforme $U \sim \text{Uniforme}(0, 1)$ al dominio de $f(x)$:

- ◊ Para el primer tramo $[0, 2]$: $X^\circ = 2U$.
- ◊ Para el segundo tramo $(2, 4]$: $X^\circ = 2U + 2$.

Por lo tanto, las condiciones de aceptación-rechazo quedan:

◊ **A-R tramo 1:**

$$U \leq X^\circ e^{-(X^\circ-1)}.$$

◊ **A-R tramo 2:**

$$U \leq (4 - X^\circ) e^{-(X^\circ-3)}.$$

Con lo anterior, se desarrollará el código para validar el método propuesto.

```
import numpy as np
import matplotlib.pyplot as plt

# --- Parametros ---
c = 0.33438326449781075
cantidad_muestras = 10000
seteo_de_semilla = 2123

# --- Semilla ---
np.random.seed(seteo_de_semilla)
```

```

# --- Funcion ---
def f(x):
    if 0 <= x <= 2:
        return 4 * c * x * np.exp(-x)
    elif 2 < x <= 4:
        return 4 * c * (4 - x) * np.exp(-x)
    else:
        return 0

# Buscamos el mximo de f(x) numricamente
x_grid = np.linspace(0, 4, 1000)
f_grid = np.array([f(xi) for xi in x_grid])
M = np.max(f_grid) * 4

def aceptacion_rechazo(cantidad_muestras):
    muestras = []
    while len(muestras) < cantidad_muestras:
        x = np.random.uniform(0, 4)
        u = np.random.uniform(0, 1)

        if u <= f(x) / (M * (1/4)):
            muestras.append(x)

    return np.array(muestras)

def densidad_teorica():
    x_vals = np.linspace(0, 4, 400)
    f_vals = [f(x) for x in x_vals]
    return x_vals, f_vals

def histograma(resultados, x_vals, f_vals):
    plt.figure(figsize=(10,6))
    plt.hist(resultados, bins=40, density=True, alpha=0.6, color='skyblue', label='Histograma de
muestras')
    plt.plot(x_vals, f_vals, 'r', lw=2, label='Densidad terica')
    plt.title('Ajuste Emprico vs. Densidad Terica')
    plt.xlabel('x')
    plt.ylabel('Densidad')
    plt.legend()
    plt.grid()
    plt.show()

if __name__ == "__main__":
    resultados = aceptacion_rechazo(cantidad_muestras)
    x_vals, f_vals = densidad_teorica()
    histograma(resultados, x_vals, f_vals)

```

Al ejecutar el código anterior, obtenemos el siguiente histograma que muestra los resultados obtenidos:

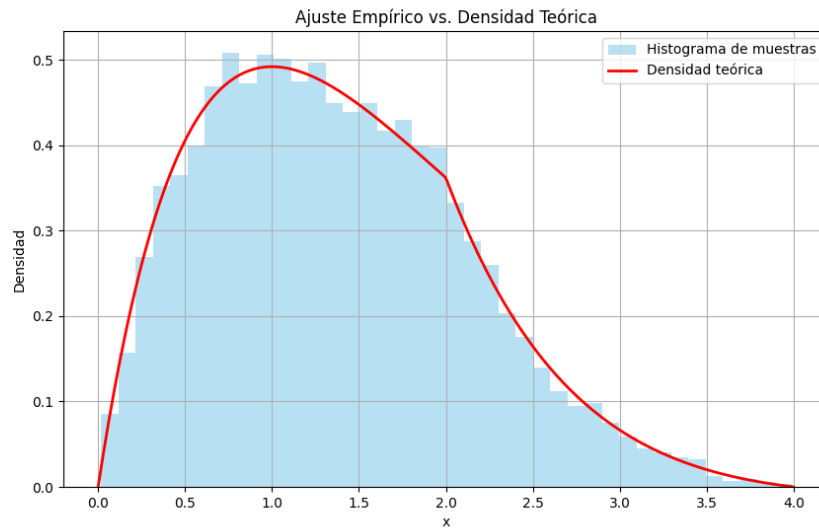


Figura 3: Histograma de los valores generados mediante el método de Aceptación-Rechazo.

El ajuste empírico obtenido mediante el método de aceptación-rechazo reproduce adecuadamente la forma general de la distribución teórica, mostrando buena concordancia en las zonas de mayor densidad de probabilidad y mayor variabilidad estadística en los extremos debido al bajo número de muestras en esas regiones. La eficiencia del método es moderada, ya que el uso de una distribución propuesta uniforme genera un número considerable de rechazos, especialmente lejos del máximo de la densidad. En conjunto, el método permite una correcta generación de muestras, aunque su eficiencia podría mejorarse utilizando propuestas más adaptadas a la forma de la densidad objetivo.